DevOps Interview Preparation Guide

Terraform | Python | GitHub Actions | Azure | AWS | Kubernetes | Docker | Ansible | Prometheus | Grafana | ELK

SECTION 1: TOP 20 INTERVIEW QUESTIONS & ANSWERS

Q1. How do you structure Terraform code for large-scale cloud infrastructure?

Category: Technical — IaC / Terraform

I follow a modular, environment-separated Terraform structure using workspaces or directory-based separation. I always store state remotely in S3/Azure Blob with state locking via DynamoDB/Azure Table Storage.

Typical directory structure:

```
terraform/
├── modules/
│   ├── networking/   # VPC, subnets, NSGs
│   ├── compute/      # EC2, AKS, EKS
│   └── storage/      # S3, Azure Storage
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── global/           # IAM, DNS, shared resources
```

Key practices:

- Use remote backend with state locking
- Enforce Terraform fmt, validate, and tfsec in CI pipeline
- Use terraform.tfvars per environment; never hardcode secrets
- Tag all resources consistently (Owner, Env, CostCenter)

Real-Time Example: At my previous company (supply chain domain), I built a Terraform module library for Azure that provisioned AKS clusters, Azure Container Registry, Key Vault, and Application Gateway. This reduced new environment setup from 3 days to 28 minutes and was reused across 6 product teams.

Q2. Walk me through a GitHub Actions CI/CD pipeline you built end-to-end.

Category: Technical — CI/CD / GitHub Actions

I designed a multi-stage GitHub Actions pipeline for a microservices application deployed to AKS. The pipeline covered build, test, security scan, and deploy stages with environment-specific approvals.

Sample workflow snippet:

```
name: Deploy to AKS
on:
```

```
push:
branches: [main]
      jobs:
build:
runs-on: ubuntu-latest
          steps:
- uses: actions/checkout@v3
- name: Build Docker Image
            run: docker build -t myapp:GITHUB_SHA .
- name: Trivy Security Scan
            uses: aquasecurity/trivy-action@master
- name: Push to ACR
            run: az acr login && docker push myregistry.azurecr.io/myapp:GITHUB_SHA
deploy:
needs: build
environment: production  # requires manual approval
          steps:
- name: Deploy to AKS
            run: |
              az aks get-credentials --name myaks --resource-group myRG
              helm upgrade --install myapp ./charts/myapp --set image.tag=GITHUB_SHA
```

Real-Time Example: Built this exact pipeline for an e-commerce platform. Reduced deployment time from 45 minutes (manual Jenkins) to 8 minutes. Added environment protection rules so production deploys require 2 approvals, cutting incidents by 40%.

Q3. How do you manage secrets and credentials in a cloud DevOps pipeline?

Category: Technical — Security / IAM

I follow a zero-hardcoded-secrets policy. My approach:

- AWS: Use AWS Secrets Manager or SSM Parameter Store. Reference via IAM roles on EC2/Lambda — no static credentials.
- Azure: Use Azure Key Vault with Managed Identities. Pods in AKS access secrets via Workload Identity.
- GitHub Actions: Store sensitive values as encrypted GitHub Secrets. Use OIDC federation to avoid long-lived credentials.

Example — OIDC-based AWS auth in GitHub Actions:

```
- name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
role-to-assume: arn:aws:iam::123456789:role/GitHubActionsRole
aws-region: us-east-1
```

Real-Time Example: At a fintech client, I replaced static AWS access keys in Jenkins jobs with OIDC-federated GitHub Actions. This eliminated 120+ static credentials and helped the team pass a SOC 2 audit with zero findings on credential management.

Q4. How do you write Python scripts for DevOps automation? Give a real example.

Category: Technical — Python Scripting

I use Python extensively for operational automation — from AWS/Azure SDK (boto3/azure-sdk) scripts to custom CLI tools and Lambda functions for event-driven automation.

Example — Python script to identify and stop idle EC2 instances:

```
import boto3
from datetime import datetime, timezone

ec2 = boto3.client('ec2', region_name='us-east-1')
cw = boto3.client('cloudwatch')

def get_idle_instances(threshold=5.0, days=7):
instances = ec2.describe_instances(Filters=[{'Name':'instance-state-name','Values':['running']}])
idle = []
for r in instances['Reservations']:
for i in r['Instances']:
iid = i['InstanceId']
metrics = cw.get_metric_statistics(
Namespace='AWS/EC2', MetricName='CPUUtilization',
Dimensions=[{'Name':'InstanceId','Value':iid}],
StartTime=datetime.now(timezone.utc).replace(days=-days),
EndTime=datetime.now(timezone.utc),
Period=86400, Statistics=['Average'])
avg = sum(p['Average'] for p in metrics['Datapoints']) / max(len(metrics['Datapoints']),1)
if avg < threshold:
idle.append(iid)
return idle
```

Real-Time Example: This script ran as a Lambda on a daily schedule via EventBridge. It identified 23 idle instances in staging environments, saving $4,200/month in EC2 costs.

Q5. How do you handle Terraform state management and state drift in production?

Category: Technical — Terraform / State Management

State management is critical in production Terraform environments. My approach:

- Remote state in S3 (with versioning) + DynamoDB state locking for AWS
- Azure: azurerm backend with Azure Blob Storage + lease-based locking
- Run terraform plan in CI for every PR to catch drift early
- Use terraform refresh and then plan -detailed-exitcode in scheduled pipelines to detect drift

Detecting and fixing drift:

```
# Detect drift
terraform plan -detailed-exitcode  # exit code 2 = drift detected

# Import manually created resources back into state
terraform import azurerm_resource_group.rg
/subscriptions/xxx/resourceGroups/my-rg

# Remove orphaned state entries
terraform state rm azurerm_storage_account.old_account
```

Real-Time Example: In production, a team manually created an Azure NSG rule during an incident. The next Terraform apply would have deleted it. I had a scheduled GitHub Actions job running terraform plan every 6 hours, which caught the drift within hours. We imported the rule into state and codified it, preventing outage.

Q6. How do you design a Kubernetes deployment strategy for zero-downtime releases?

Category: Technical — Kubernetes / Docker

I use a combination of Rolling Updates, Blue-Green, and Canary deployments depending on risk tolerance:

- Rolling Update (default): Gradual pod replacement — suitable for most microservices.
- Blue-Green: Two identical environments; switch traffic via ingress/service selector. Full rollback in seconds.
- Canary: Route a percentage of traffic to new version. I use Argo Rollouts or Istio for this.

Rolling update config with readiness probes:

```
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1
maxUnavailable: 0   # Zero downtime guaranteed
readinessProbe:
httpGet:
path: /health
```

port: 8080
initialDelaySeconds: 10
periodSeconds: 5

Real-Time Example: For a supply chain order management service on AKS, I implemented canary deployments using Argo Rollouts. We initially rolled out to 10% traffic, monitored error rate on Grafana, and auto-promoted after 30 minutes if error rate stayed below 0.1%. This prevented 3 bad releases from reaching production in one quarter.

Q7. How do you implement cloud cost optimization in AWS or Azure?

Category: Technical — Cloud / Cost Optimization

Cost optimization is a continuous practice. My framework:

- Right-sizing: Analyze CloudWatch/Azure Monitor metrics to downsize over-provisioned instances.
- Reserved Instances / Savings Plans: Automate RI recommendations using Cost Explorer APIs.
- Auto-scaling: HPA/KEDA for Kubernetes; ASGs for EC2; Azure VMSS for VMs.
- Idle resource cleanup: Python + boto3/azure-sdk scripts to detect and schedule deletion.
- S3/Blob lifecycle policies: Move old data to cheaper tiers automatically.

Real-Time Example: At a logistics company, I ran a 3-month cost optimization initiative. I identified 40 unattached EBS volumes (Python script), converted 15 dev environments to spot instances, and set up S3 Intelligent-Tiering. Total savings: $28,000/month — a 35% reduction. Presented the findings in a FinOps dashboard on Grafana.

Q8. How do you set up monitoring and alerting for a production Kubernetes cluster?

Category: Technical — Monitoring / Observability

I implement the full observability stack: metrics (Prometheus + Grafana), logs (ELK/Loki), and traces (Jaeger/OpenTelemetry).

- Deploy kube-prometheus-stack via Helm for Prometheus + Grafana + AlertManager
- Configure ServiceMonitors for application metrics scraping
- Set up AlertManager with PagerDuty/Slack routing rules
- ELK: Fluent Bit as DaemonSet to ship container logs to Elasticsearch; Kibana for visualization

Sample Prometheus alert rule:

```
groups:
- name: pod-alerts
  rules:
```

- alert: PodCrashLooping

```
    expr: rate(kube_pod_container_status_restarts_total[5m]) > 0
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: 'Pod {{ $labels.pod }} is crash looping'
```

Real-Time Example: Set up full Prometheus+Grafana+ELK stack for a 200-node AKS cluster. Created 15 custom dashboards. Configured tiered AlertManager routing — P1 alerts hit PagerDuty, P2 goes to Slack. MTTR dropped from 45 minutes to 12 minutes after rollout.

Q9. Describe your experience with Ansible for configuration management.

Category: Technical — Configuration Management

I use Ansible for idempotent configuration management, application deployment, and patch management. I organize playbooks into roles following Ansible Galaxy standards.

Sample role structure and task:

```
roles/nginx/tasks/main.yml:
```

- name: Install nginx

apt:

```
    name: nginx
```

state: present

update_cache: yes

- name: Deploy nginx config from template

template:

src: nginx.conf.j2

dest: /etc/nginx/nginx.conf

notify: Restart nginx

- name: Ensure nginx is started

service:

```
    name: nginx
```

state: started

enabled: yes

Real-Time Example: Used Ansible to manage configuration of 300+ EC2 instances across 4 environments. Replaced manual SSH-based setup with automated role-based provisioning. Reduced configuration drift incidents to zero and cut onboarding time for new services from 2 days to 2 hours.

Q10. How do you design and implement IAM and cloud security best practices?

Category: Technical — Cloud Security / IAM

I follow the principle of least privilege religiously. My security framework:

- AWS: IAM roles per service (never shared). SCPs at org level. CloudTrail + GuardDuty + Security Hub enabled across all accounts.
- Azure: RBAC with custom roles. Managed Identities for service-to-service auth. Azure Policy for compliance enforcement.
- Network: Private endpoints for all PaaS services. NSGs/Security Groups with deny-all default. WAF in front of public endpoints.
- Secrets: Never in code or env vars — always Key Vault / Secrets Manager with IAM-based access.
- Scanning: tfsec and checkov in CI to catch misconfigs before deploy.

Real-Time Example: Led a cloud security hardening project for an Azure environment before a PCI-DSS audit. Implemented Azure Policy (deny public IPs for VMs), enabled Defender for Cloud, and added checkov to all IaC pipelines. Reduced CIS benchmark failures from 147 to 3 in 6 weeks. Passed audit with zero major findings.

Q11. SCENARIO: Production deployment fails mid-rollout. What do you do?

Category: Scenario-Based — Incident Response

Step-by-step incident response:

1. IMMEDIATE: Assess blast radius. How many users/services are affected? Is it a full outage or partial?
2. ROLLBACK FIRST: Don't debug in prod. Trigger rollback immediately.
3. COMMUNICATE: Post to incident channel. Update status page. Notify stakeholders.
4. ROOT CAUSE: Once stable, analyze logs, metrics, and changes deployed.
5. FIX FORWARD: Apply fix in staging, test, re-deploy with extra monitoring.
6. POST-MORTEM: Blameless post-mortem within 48 hours. Document timeline, root cause, and preventive actions.

Rollback commands — Kubernetes:

```
# Check rollout history
kubectl rollout history deployment/myapp -n production
# Rollback to previous version
kubectl rollout undo deployment/myapp -n production
# Monitor rollback
kubectl rollout status deployment/myapp -n production
```

Real-Time Example (STAR): S: A payment service deployment to AKS caused 500 errors for 15% of users. T: Minimize downtime and prevent data loss. A: Ran kubectl rollout undo, confirmed all pods healthy in 3 minutes, notified team via PagerDuty, and opened an incident channel. Post-incident, found a missing env variable. Added config validation to the pre-deploy stage. R: Incident lasted 7 minutes total. No data loss. New validation prevented 2 similar issues in the next quarter.

Q12. SCENARIO: Terraform plan shows resources will be destroyed in prod. How do you handle it?

Category: Scenario-Based — Terraform / Risk Management

This is a critical situation. My response:

- NEVER run terraform apply blindly when destroy actions appear in plan.
- Understand WHY: Is it intended? Is it a state drift? Is it a resource rename?
- Use lifecycle rules to protect critical resources:

```
resource "azurerm_postgresql_server" "db" {
lifecycle {
prevent_destroy = true      # Blocks accidental destruction
create_before_destroy = true
}
}
```

- Use targeted apply if only specific resources need updating: terraform apply -target=azurerm_app_service.web
- In CI pipeline: enforce a manual approval gate on any plan that contains destroy actions.

Real-Time Example: A team member renamed an RDS identifier in Terraform, which showed destroy+create in the plan — this would have caused data loss. My pipeline's destroy-detection script flagged it, blocked auto-apply, and sent an alert. We used: terraform state mv aws_db_instance.old aws_db_instance.new — zero downtime, zero data loss.

Q13. SCENARIO: Cloud costs suddenly spike 200% overnight. How do you investigate?

Category: Scenario-Based — Cloud Cost / Troubleshooting

My systematic investigation:

1. Check AWS Cost Explorer / Azure Cost Analysis — filter by service, region, and time to find the spike source.
2. Check CloudTrail / Azure Activity Log — what was deployed or changed in the last 24 hours?
3. Look for runaway autoscaling, forgotten large instances, data transfer charges, or NAT Gateway overuse.
4. Set up Cost Anomaly Detection (AWS) or Budget Alerts (Azure) for next time.

Python — detect new large instances launched in last 24h:

```
import boto3
from datetime import datetime, timedelta, timezone
```

```
ec2 = boto3.client('ec2')
response = ec2.describe_instances(Filters=[
{'Name': 'launch-time', 'Values': [(datetime.now(timezone.utc)-
timedelta(days=1)).strftime('%Y-%m-%dT*')]}
])
for r in response['Reservations']:
for i in r['Instances']:
print(i['InstanceId'], i['InstanceType'], i['LaunchTime'])
```

Real-Time Example: A developer spun up a p3.16xlarge GPU instance ($24/hour) for a test and forgot about it over a long weekend. My cost anomaly alert fired within 4 hours. The instance was stopped within 5 hours of launch, saving $2,800.

Q14. How do you implement auto-scaling in Kubernetes and cloud environments?

Category: Technical — Kubernetes / Scaling

- HPA (Horizontal Pod Autoscaler): Scale pods based on CPU/memory or custom metrics via Prometheus Adapter.
- VPA (Vertical Pod Autoscaler): Automatically right-size pod resource requests.
- KEDA: Event-driven autoscaling based on external triggers (queue length, Kafka lag, etc.).
- Cluster Autoscaler: Automatically add/remove nodes based on pending pods.

HPA with custom Prometheus metric:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: order-service
minReplicas: 2
maxReplicas: 20
metrics:
- type: Pods
pods:
metric:
name: orders_per_second
target:
type: AverageValue
averageValue: 100
```

Real-Time Example: For a retail client's peak sale event (Diwali sale), I configured KEDA to scale order-processing pods based on RabbitMQ queue depth. During the event, pods scaled

from 5 to 87 automatically within minutes, processing 3.2 million orders with zero manual intervention and p99 latency under 200ms.

Q15. How do you manage multi-environment (dev/staging/prod) infrastructure in Terraform?

Category: Technical — Terraform / Environments

I use two approaches depending on team size and complexity:

- Directory-based separation: Separate terraform/ directories per environment with shared modules. Best for environments with significantly different configs.
- Terraform Workspaces: Single codebase, multiple state files. Best for environments with identical structure but different sizing.

Environment-specific variable files:

```
# environments/prod/terraform.tfvars
environment    = "prod"
aks_node_count = 10
aks_node_size  = "Standard_D4s_v3"
enable_ha      = true

# environments/dev/terraform.tfvars
environment    = "dev"
aks_node_count = 2
aks_node_size  = "Standard_B2s"
enable_ha      = false
```

In GitHub Actions, the environment is determined by branch:

```
- name: Terraform Apply
  run: terraform apply -var-file=environments/BRANCH_NAME/terraform.tfvars -auto-approve
```

Real-Time Example: Managed 4 environments for a logistics platform on Azure using directory-based Terraform. Each environment shared 12 reusable modules. Production had stricter NSG rules, Private Endpoints enforced via Azure Policy, and geo-redundant storage — all handled via tfvars overrides.

Q16. How do you ensure high availability and disaster recovery in cloud infrastructure?

Category: Technical — Cloud Architecture / HA & DR

HA and DR require design-time decisions, not afterthoughts. My approach:

- Multi-AZ deployments: Spread compute, databases, and load balancers across availability zones.
- RDS Multi-AZ / Azure SQL Geo-Replication: Automatic failover for databases.
- S3 Cross-Region Replication / Azure Geo-Redundant Storage for critical data.
- Define RPO (Recovery Point Objective) and RTO (Recovery Time Objective) upfront.
- Chaos engineering: Periodically run Game Days to test failover procedures.

Key Terraform snippet for multi-AZ AKS:

```
resource "azurerm_kubernetes_cluster_node_pool" "system" {
zones      = ["1", "2", "3"]   # Spread across AZs
node_count = 3
vm_size    = "Standard_D4s_v3"
}
```

Real-Time Example: Designed HA architecture for a supply chain visibility platform on AWS: Multi-AZ EKS, RDS PostgreSQL Multi-AZ, S3 with CRR to DR region, and Route53 health checks with failover routing. During an actual AZ outage (US-EAST-1B failure), the system automatically failed over within 90 seconds — zero manual intervention.

Q17. BEHAVIORAL: Tell me about a time you led a critical infrastructure migration.

Category: Behavioral — Leadership / STAR Format

Situation: Our company needed to migrate a monolithic application from on-premise VMware to AWS within 3 months to avoid a $500K data center renewal cost.

Task: I was assigned as the migration lead — responsible for infrastructure design, execution plan, risk mitigation, and team coordination (5 engineers).

Action: I started with a 2-week discovery phase using AWS Application Discovery Service to map all 45 servers and dependencies. I designed a phased lift-and-shift approach using AWS MGN (Application Migration Service), followed by re-platforming to containers in Phase 2. I set up a parallel environment in AWS, ran dark-launch testing for 3 weeks, and created a runbook for each service migration with rollback steps.

Result: Completed migration 2 weeks ahead of schedule. Zero production incidents during cutover. Phase 2 re-platforming to EKS reduced compute costs by 45%. The migration saved $470K in year 1 and became a template for 2 more migrations across the organization.

Q18. BEHAVIORAL: Tell me about a time you resolved a major production incident.

Category: Behavioral — Incident Handling / STAR Format

Situation: A critical API gateway in production went down at 2 AM, causing 100% failure of order processing for a retail client during a flash sale. Revenue impact was $50,000 per minute.

Task: As the on-call DevOps engineer, I had to diagnose and restore service as quickly as possible while keeping stakeholders informed.

Action: I joined the incident bridge immediately. Checked Grafana dashboards — saw CPU spike to 100% on the API gateway pods. Checked Kibana logs — found a memory leak in a newly deployed version. Immediately rolled back using kubectl rollout undo. While pods were recovering, I drafted a status update for stakeholders. Service was restored in 11 minutes. Led a blameless post-mortem, identified the missing load test in staging, and added memory profiling and load testing as mandatory gates in the CI/CD pipeline.

Result: Service restored in 11 minutes. Total revenue impact contained. Post-mortem actions eliminated similar incidents for 14+ months. I was recognized with a spot award for the response.

Q19. BEHAVIORAL: How do you handle disagreements with developers about infrastructure decisions?

Category: Behavioral — Collaboration / Communication

I approach disagreements by focusing on data, business outcomes, and mutual education — not on being right.

- I first listen fully to understand the developer's requirements and constraints.
- I explain the infrastructure implications (security, cost, reliability) with concrete examples — not just 'best practices.'
- I document the pros and cons of each approach and let the data drive the decision.
- When still unresolved, I involve an architect or senior stakeholder with full context presented objectively.

Real-Time Example: A dev team wanted to use a single shared database for all microservices to 'save time.' I documented the risks (noisy neighbor problem, blast radius, schema coupling) with examples from past incidents. I proposed a middle ground: a shared PostgreSQL server with separate databases per service, managed by Terraform, meeting both the cost goal and the isolation requirement. The team adopted it and it became the company standard.

Q20. How do you stay current with DevOps tools and cloud trends?

Category: Behavioral / Cultural Fit

- Daily: Follow AWS/Azure/HashiCorp release notes, CNCF newsletter, and DevOps Weekly.
- Learning: Certifications in AWS Solutions Architect, CKA (Certified Kubernetes Administrator), and HashiCorp Terraform Associate.
- Practice: I maintain a personal GitHub lab repository where I experiment with new tools (recently: OpenTofu, Argo CD GitOps, Karpenter for node provisioning).

- Community: I contribute to open-source Terraform modules and attend local DevOps meetups.
- At work: I run internal 'DevOps Friday' tech talks — 30-minute sessions where the team shares new tools or post-incident learnings.

Example: I recently migrated our team from Terraform Cloud to Atlantis + GitHub Actions for plan/apply workflows — a tool I learned about from the CNCF Slack community. This gave us better audit trails, cost $0 vs $300/month for Terraform Cloud, and integrated seamlessly with our existing GitHub-based workflow.

SECTION 2: TRICKY QUESTIONS & HOW TO HANDLE THEM

T1. "You have 8.5 years experience but you don't seem to know X deeply."

Category: Handling Tough Questions

Stay calm. Acknowledge it honestly, then pivot to strength and learning attitude.

"You're right — I haven't used X in production, but I have strong foundational knowledge of it and have worked with Y which solves similar problems. Given my track record of picking up tools quickly, I'm confident I can get productive with X within 2-3 weeks. I've already started exploring it — [mention what you've done]."

T2. "Terraform vs Pulumi — why should we use Terraform?"

Category: Tool Comparison

Don't be a zealot — show balanced thinking:

"Both have merits. Terraform has the largest ecosystem, mature state management, and is the industry standard for multi-cloud IaC. Pulumi is excellent for teams that want to use general-purpose languages like Python or TypeScript. I would always let team skill and project requirements drive the tool decision."

T3. "What's the biggest mistake you've made in your DevOps career?"

Category: Self-Awareness

Be honest, show self-awareness and learning:

"Early in my career, I ran terraform apply in the wrong environment — staging config in production — due to a missing env check. It caused a 20-minute partial outage. From that incident, I implemented environment-tagged state backends, branch-based pipeline triggers, and mandatory plan review steps. I now always have 2 safeguards before any apply in production. That mistake made me design much more defensively."

T4. "How do you handle a situation where management asks for a shortcut that compromises security?"

Category: Integrity / Business Awareness

"I'd present the risk clearly — in business terms, not just technical ones. For example: 'If we skip encryption, we risk a breach that could cost $X in fines and $Y in customer trust.' I'd propose a risk-adjusted alternative that meets the timeline while preserving security essentials. If still overruled, I'd ensure the risk is documented and accepted in writing. I've never compromised on security posture without documented, conscious risk acceptance."

T5. "Where do you see yourself in 5 years?"

Category: Career Goals

"I see myself growing into a Principal DevOps Engineer or Cloud Architect role — designing infrastructure strategies at an organizational level, mentoring teams, and driving platform engineering initiatives. I'm looking for an environment where I can grow toward that goal while delivering real impact."

SECTION 3: QUESTIONS TO ASK THE INTERVIEWER

About the Role & Team

- What does the current infrastructure setup look like — what's the ratio of AWS vs Azure usage today?
- How large is the DevOps/platform team, and how is it structured (centralized platform vs embedded)?
- What are the biggest infrastructure or DevOps pain points the team is trying to solve in the next 6 months?

About Technology & Practices

- What's the current state of your Terraform adoption — are you fully on IaC or still in transition?
- How do you handle Terraform state management at scale — do you use Terraform Cloud, Atlantis, or self-hosted?
- What's your current Kubernetes setup — EKS, AKS, or self-managed? What's the cluster count and scale?
- How mature is your observability stack — are you using a centralized monitoring platform?

About Culture & Growth

- How does the team approach blameless post-mortems and continuous improvement?
- What does the on-call rotation look like, and how are incidents managed?
- What does a successful first 90 days look like for someone in this role?
- Are there opportunities to contribute to open-source or internal developer tooling initiatives?